

MODULE 08

Unsupervised Learning - Clustering

AI with Data Science - Professional Learning Series

ARSHITH LMS | Professional Edition

Unsupervised Learning - Clustering

What is Unsupervised Learning?

Unsupervised learning finds hidden patterns in unlabelled data.

Clustering: group similar data points

Dimensionality Reduction: compress features

Anomaly Detection: find unusual data points

Association Rules: find frequent patterns

Applications: customer segmentation, topic modelling, image compression

K-Means Clustering

```
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt

# Scale features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Find optimal k with Elbow method
wcss = []
for k in range(1, 11):
    km = KMeans(n_clusters=k, random_state=42)
    km.fit(X_scaled)
    wcss.append(km.inertia_)
plt.plot(range(1, 11), wcss, marker='o')

# Apply K-Means
km = KMeans(n_clusters=4, random_state=42)
df['cluster'] = km.fit_predict(X_scaled)
```

Hierarchical Clustering

```
from sklearn.cluster import AgglomerativeClustering
from scipy.cluster.hierarchy import dendrogram, linkage
import matplotlib.pyplot as plt

# Dendrogram to find clusters
Z = linkage(X_scaled, method='ward')
plt.figure(figsize=(12, 5))
dendrogram(Z, truncate_mode='level', p=5)
plt.show()

# Apply hierarchical clustering
agg = AgglomerativeClustering(n_clusters=4, linkage='ward')
```

```
df['cluster'] = agg.fit_predict(X_scaled)
```

DBSCAN Clustering

```
from sklearn.cluster import DBSCAN

# DBSCAN - density based, finds outliers automatically
dbscan = DBSCAN(eps=0.5, min_samples=5)
df['cluster'] = dbscan.fit_predict(X_scaled)

# -1 = noise/outlier
print('Clusters found:', df['cluster'].nunique() - 1)
print('Outliers:', (df['cluster'] == -1).sum())

# Advantages over K-Means:
# No need to specify k
# Finds arbitrary shapes
# Handles outliers
```

Cluster Evaluation

```
from sklearn.metrics import silhouette_score, davies_bouldin_score

# Silhouette score: -1 to 1 (higher=better)
silhouette = silhouette_score(X_scaled, labels)
print(f'Silhouette Score: {silhouette:.3f}')

# Davies-Bouldin index (lower=better)
db_score = davies_bouldin_score(X_scaled, labels)
print(f'Davies-Bouldin: {db_score:.3f}')

# Business interpretation:
# Profile each cluster
df.groupby('cluster')[features].mean()
```